

EDUGENIE

Google Gemini Powered Learning Assistant

PROJECT REPORT

Team Leader: Sri Dharshini

Team Members

Nirmal

Sriram

Ashwin

Department / Institution: _____

Academic Year: 2026–2027

CERTIFICATE

This is to certify th
as a team project
describes the design, implementation and testing of an AI-assisted educational web application using a
FastAPI backend and a browser-based frontend.

resented
e work

Guide / Mentor: _____ Signature: _____

Head of Department: _____ Signature: _____

DECLARATION

We declare that this project report presents our proposed implementation and documentation of EduGenie, an AI-powered learning assistant. The report expands the supplied project material with newly written explanations, example implementations, data-flow descriptions, test cases and diagrams for academic project documentation.

Team L Team Leader: Sri Dharshini

Membe Members: Nirmal, Sriram, Ashwin

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty, mentor and department for their guidance and encouragement during the development of this project. We also acknowledge the software and AI technologies that make it possible to build interactive educational applications with relatively lightweight infrastructure. The project helped us understand backend API development, frontend integration, prompt design, JSON processing, testing and responsible use of generative AI.

ABSTRACT

EduGenie is a web-based learning assistant designed to help learners understand academic topics through interactive AI features. The supplied project describes five core capabilities: question answering, simplified explanation, quiz generation, summarization and personalized learning recommendations. The system uses FastAPI for backend services, HTML/CSS for the web interface, a cloud AI model for several generative tasks, and a lightweight local model for concept explanation.

This expanded project design organizes these capabilities into modular services so that each feature can be tested independently and improved without rewriting the complete application. The report also introduces validation, error handling, request/response schemas, testing strategy, security considerations, logging concepts and future extensions such as multilingual learning, progress dashboards, document input and mobile access.

Keywords: Generative AI, Gemini, FastAPI, Python, Educational Technology, Quiz Generation, Summarization, Personalized Learning

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Existing and Proposed System
5. System Requirements
6. System Architecture
7. Module Design
8. Database and Data Design
9. API Design
10. Coding and Implementation
11. Frontend Design
12. Prompt Engineering
13. Algorithms and Pseudocode
14. Testing and Validation
15. Security and Responsible AI
16. Deployment
17. Results and Discussion
18. Advantages and Limitations
19. Future Enhancements
20. Conclusion
21. References

1. INTRODUCTION

Digital learning environments often contain large amounts of information, but learners may still need quick explanations, revision material and structured guidance. EduGenie addresses this gap by combining a simple web interface with generative AI services.

The supplied source identifies EduGenie as a lightweight AI-powered educational assistant. It supports questions, simplified explanations, quizzes, summaries and learning recommendations. The original architecture uses FastAPI and HTML/CSS, with Gemini used for question answering, summarization, quiz generation and learning paths, while LaMini-Flan-T5 is described for concept explanation.

1.1 Project Scope

- Interactive educational question answering
- Simple explanations for difficult concepts
- Automatic multiple-choice quiz generation
- Condensing long passages into revision notes
- Topic-based beginner-to-advanced learning paths
- REST API integration between browser and backend
- Modular Python implementation for maintainability

1.2 Target Users

- School and college students
- Self-learners
- Teachers preparing quick revision activities
- Developers learning how to integrate generative AI into web applications

2. PROBLEM STATEMENT

Learners frequently move between search engines, notes, quiz websites and tutorials to complete one study task. This creates fragmented workflows. A learner may understand an answer but still need a simpler explanation, a short summary, practice questions and a learning sequence.

EduGenie aims to place these related learning activities behind one interface. Instead of treating AI as a general chat box only, the system provides task-specific operations with predictable inputs and outputs.

2.1 Proposed Problem Solution

Problem	EduGenie Response
Complex academic text	Simplification module
Doubt while studying	Q&A module
Need for practice	Quiz module
Long material	Summary module
Unclear study sequence	Learning Path module

3. OBJECTIVES

- Develop a modular AI-powered learning assistant.
- Provide a clean browser interface for common study operations.
- Expose separate FastAPI endpoints for each learning function.
- Use structured prompts to make AI outputs useful for learners.
- Validate quiz output so it can be safely rendered by the frontend.
- Provide clear errors when AI responses or requests are invalid.
- Design the project so additional modules can be added later.

3.1 Measurable Functional Goals

Goal	Expected Behaviour
Q&A	Returns a concise answer to a learner question
Explain	Converts a topic into beginner-friendly explanation
Quiz	Produces MCQs with options and answers
Summarize	Returns key ideas in shorter form
Recommend	Produces a staged learning path

4. EXISTING AND PROPOSED SYSTEM

4.1 Existing Learning Workflow

In a conventional workflow, students search separately for explanations, summaries, questions and tutorials. The quality and format of results may vary, and the learner has to organize the material manually.

4.2 Proposed EduGenie Workflow

EduGenie provides a single task selector and input area. The frontend sends a POST request to the appropriate FastAPI endpoint. The backend validates the request, builds a task-specific prompt, invokes the configured model, normalizes the response and sends the result back to the browser.

High-Level Learning Workflow

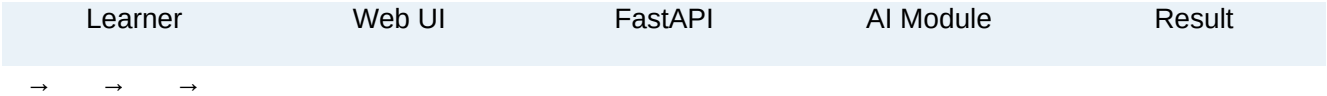


Figure: High-Level Learning Workflow

4.3 Comparative View

Aspect	Conventional Workflow	EduGenie
Interface	Multiple sites	Single interface
Task switching	Manual	Task selector
Quiz format	Find/create separately	Generated by module
Learning path	Manual research	AI recommendation
Architecture	External services	Modular API design

5. SYSTEM REQUIREMENTS

5.1 Hardware

- Modern laptop or desktop computer
- At least 4 GB RAM for basic development; more is useful for local models
- Internet connection for cloud AI requests
- Browser such as Chrome, Edge or Firefox

5.2 Software

- Python 3.10+
- FastAPI
- Uvicorn
- Jinja2
- HTML5 and CSS3
- Configured Google Gemini API access
- Optional local transformer model for explanation

5.3 Installation Commands

Environment Setup

```
python -m venv .venv
```

```
# Windows:
```

```
.venv\Scripts\activate
```

```
# Linux/macOS:
```

```
source .venv/bin/activate
```

```
pip install fastapi uvicorn jinja2 python-dotenv
```

6. SYSTEM ARCHITECTURE

The architecture follows a layered pattern: presentation, API routing, service modules and AI/model integration. This separation keeps frontend concerns away from prompt and model logic.

Layered Architecture

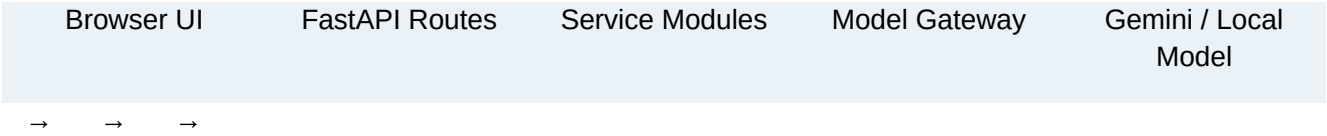


Figure: Layered Architecture

6.1 Request Flow

- User selects a task and enters text.
- Frontend sends JSON or form data to the relevant endpoint.
- FastAPI validates the request.
- The service module creates a structured prompt.
- The AI provider generates a response.
- The service validates and normalizes the response.
- FastAPI returns JSON or text.
- Frontend displays the result.

6.2 Failure Flow

Error Handling Flow

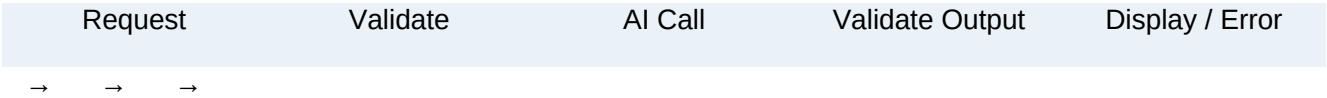


Figure: Error Handling Flow

7. MODULE DESIGN

7.1 Question Answering Module

The Q&A module accepts a natural-language question and requests a direct educational answer. Prompt instructions can request clarity, concise language and appropriate examples.

7.2 Explanation Module

The explanation module focuses on reducing complexity. A topic can be converted into a definition, core idea, example and short recap. The source project associates this function with a lightweight LaMini-Flan-T5 model.

7.3 Quiz Module

The quiz module generates multiple-choice questions from supplied text or a topic. A structured JSON format is preferred because the frontend can render question cards programmatically.

7.4 Summary Module

The summary module removes repetition while preserving important concepts. It can be configured for a target length or bullet-point output.

7.5 Learning Path Module

The recommendation module produces a progression from beginner concepts to more advanced topics, with suggested resources and a logical study order.

8. DATABASE AND DATA DESIGN

The initial EduGenie design can operate without a permanent database. For a production-oriented extension, a database can store user profiles, learning history, quiz attempts and progress. The following schema is a proposed extension rather than a requirement of the supplied source.

Entity	Important Fields	Purpose
User	user_id, name, created_at	Learner profile
Session	session_id, user_id, started_at	Interaction grouping
Activity	activity_id, type, input, output	Learning history
QuizAttempt	quiz_id, score, timestamp	Assessment tracking
Progress	user_id, topic, level, streak	Learning analytics

8.1 Example SQL

Proposed SQLite Schema

```
CREATE TABLE users (  
    user_id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    created_at TEXT NOT NULL  
);  
  
CREATE TABLE activities (  
    activity_id INTEGER PRIMARY KEY,  
    user_id INTEGER,  
    activity_type TEXT NOT NULL,  
    input_text TEXT NOT NULL,  
    output_text TEXT,  
    created_at TEXT NOT NULL  
);
```

9. API DESIGN

The supplied project lists REST endpoints for /qa, /explain, /quiz, /summarize and /learn/recommendations. The following request/response contracts provide a clearer implementation model.

Endpoint	Method	Input	Output
/qa	POST	question	answer
/explain	POST	topic	explanation
/quiz	POST	passage	questions
/summarize	POST	text	summary
/learn/recommendations	POST	topic	learning path

9.1 Example Request

Q&A JSON

```
{  
  "question": "Explain photosynthesis in simple words."  
}
```

9.2 Example Response

Q&A Response

```
{  
  "answer": "Photosynthesis is the process by which green plants..."  
}
```

10. CODING AND IMPLEMENTATION

10.1 FastAPI Application

main.py

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="EduGenie API")

class TextRequest(BaseModel):
    text: str

class QuestionRequest(BaseModel):
    question: str

@app.get("/")
def home():
    return {"message": "EduGenie is running"}

@app.post("/qa")
def qa(req: QuestionRequest):
    return {"answer": generate_answer(req.question)}

@app.post("/summarize")
def summarize(req: TextRequest):
    return {"summary": generate_summary(req.text)}
```

10.2 AI Service Layer

ai_service.py

```
import os

def generate_answer(question: str) -> str:
    prompt = f"""
    You are an educational assistant.
    Answer the following question clearly for a student:
    {question}
    """
    return call_model(prompt)

def generate_summary(text: str) -> str:
    prompt = f"""
    Summarize the following educational text.
    Keep the main facts and use simple language.
    TEXT:
    {text}
    """
    return call_model(prompt)
```

10.3 Quiz JSON Normalization

quiz_module.py

```
import json
```

```
def clean_json_block(raw: str):
    raw = raw.strip()
    if raw.startswith("```"):
        raw = raw.replace("```json", "").replace("```", "").strip()
    return json.loads(raw)

def validate_quiz(items):
    if not isinstance(items, list):
        raise ValueError("Quiz must be a list")
    for item in items:
        if len(item.get("options", [])) != 4:
            raise ValueError("Each question needs four options")
        if "answer" not in item:
            raise ValueError("Missing correct answer")
    return items
```

11. FRONTEND DESIGN

11.1 HTML Interface

templates/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>EduGenie</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <main class="container">
    <h1>EduGenie</h1>
    <p>AI-powered learning assistant</p>

    <select id="task">
      <option value="qa">Ask a Question</option>
      <option value="explain">Explain a Topic</option>
      <option value="quiz">Generate Quiz</option>
      <option value="summarize">Summarize</option>
      <option value="recommend">Learning Path</option>
    </select>

    <textarea id="input" placeholder="Enter your topic or text"></textarea>
    <button onclick="runTask()">Submit</button>
    <section id="result"></section>
  </main>
</body>
</html>
```

11.2 CSS

static/style.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f4f7fb;
}
.container {
  max-width: 800px;
  margin: 40px auto;
  padding: 24px;
  background: white;
  border-radius: 12px;
}
textarea {
  width: 100%;
  min-height: 180px;
  margin: 12px 0;
}
button {
  padding: 10px 18px;
  cursor: pointer;
}
```


12. PROMPT ENGINEERING

Prompt engineering is important because the same model can produce different output structures depending on instructions. EduGenie can use role, task, constraints and output-format sections.

12.1 Q&A Prompt Template

Prompt

ROLE: You are a patient educational tutor.

TASK: Answer the student's question.

RULES:

- Use simple language.
- Explain the core idea first.
- Add one short example when useful.
- Do not invent facts when the input is uncertain.

QUESTION: {question}

12.2 Quiz Prompt Template

Prompt

Create 3 multiple-choice questions from the passage.

Return ONLY valid JSON:

```
[
  {
    "question": "...",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }
]
```

Passage: {text}

12.3 Learning Path Prompt

Prompt

Create a learning path for: {topic}

Include:

1. Beginner foundations
2. Intermediate concepts
3. Advanced concepts
4. Practice activities
5. Suggested resource types

Use a clear numbered sequence.

13. ALGORITHMS AND PSEUDOCODE

13.1 Q&A Algorithm

Pseudocode

```
START
Read question
Validate question is not empty
Build educational prompt
Send prompt to AI service
Receive response
If response is valid:
    return answer
Else:
    return error message
END
```

13.2 Quiz Algorithm

Pseudocode

```
START
Read passage
Build JSON-output prompt
Request questions from AI
Clean Markdown wrappers if present
Parse JSON
Validate number of options
Validate answer field
Return quiz
END
```

13.3 Summary Algorithm

Pseudocode

```
START
Read source text
Check minimum input length
Build summary prompt
Generate response
Check response exists
Return concise summary
END
```

13.4 Learning Path Algorithm

Pseudocode

```
START
Read topic
Ask AI for staged progression
Separate beginner, intermediate and advanced sections
Attach practice/resource suggestions
Return formatted learning path
```

END

14. TESTING AND VALIDATION

Testing checks both the API mechanics and the usefulness of generated educational output. AI outputs are probabilistic, so validation should focus on structure, safety, completeness and consistency rather than assuming every generated answer is automatically correct.

Test ID	Scenario	Expected Result
T01	GET /	Service status message
T02	Empty Q&A	Validation/error response
T03	Valid question	Answer returned
T04	Quiz generation	Questions contain four options
T05	Malformed quiz JSON	Readable error; no crash
T06	Long summary text	Shorter summary returned
T07	Learning path	Ordered progression returned
T08	Unknown endpoint	HTTP 404
T09	Browser submit	Result appears in UI
T10	API unavailable	Frontend shows useful error

14.1 Test Execution Checklist

- Start the Uvicorn server.
- Open the browser interface.
- Run one request for each task.
- Inspect the raw API response.
- Check frontend formatting.
- Repeat with empty and unusually long inputs.
- Record errors and improve validation.

15. SECURITY AND RESPONSIBLE AI

An educational AI application should protect API credentials and avoid treating generated responses as guaranteed truth. API keys should be stored in environment variables rather than committed to source code.

- Never place a private API key directly in frontend JavaScript.
- Use environment variables for secrets.
- Validate request size and content.
- Handle provider failures without exposing sensitive configuration.
- Avoid storing learner data unless there is a clear purpose and appropriate protection.
- Encourage users to verify important academic facts with trusted materials.
- Use structured output validation before rendering AI-generated data.

15.1 Environment Configuration

.env example

```
GEMINI_API_KEY=your_key_here  
MODEL_NAME=your_configured_model
```

15.2 Responsible Educational Use

EduGenie is best treated as a learning companion rather than an unquestionable authority. Students should use generated explanations to support understanding and should consult textbooks, teachers or authoritative sources when accuracy is important.

16. DEPLOYMENT

16.1 Local Run

Command

```
uvicorn main:app --reload
# Open:
http://127.0.0.1:8000
```

16.2 Suggested Deployment Structure

Deployment

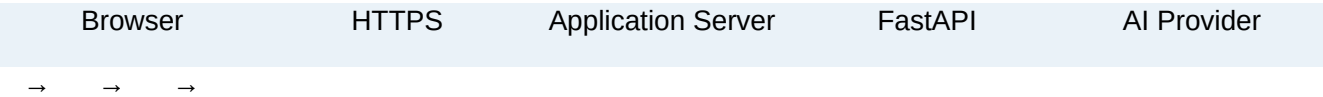


Figure: Deployment

16.3 Production Considerations

- Use HTTPS.
- Store secrets in managed environment configuration.
- Disable verbose debug output.
- Add request limits and logging.
- Use a process manager for reliable service operation.
- Monitor provider errors and response latency.

17. RESULTS AND DISCUSSION

The proposed EduGenie implementation provides a unified workflow for common learning tasks. The supplied project demonstrates the intended feature set and describes local execution through Uvicorn. In the expanded design, each operation has a dedicated endpoint and validation layer.

Feature	User Input	System Output
Q&A	Question	Educational answer
Explain	Topic	Simplified explanation
Quiz	Passage/topic	MCQs with options
Summary	Long text	Concise revision text
Learning Path	Topic	Staged learning plan

17.1 Example User Journey

A learner enters “Database normalization” and selects Explain. After reading the result, the learner can switch to Quiz and submit the same topic for practice. The learner can then request a Learning Path to identify prerequisite and advanced topics. This creates a simple study loop: understand → practice → organize next steps.

18. ADVANTAGES AND LIMITATIONS

18.1 Advantages

- Single interface for several study tasks.
- Modular backend is easier to maintain.
- Structured quiz output supports automated rendering.
- Cloud AI can provide broad language understanding.
- Lightweight frontend is easy to run and modify.
- Future modules can be added without redesigning the entire application.

18.2 Limitations

- Generated answers can contain inaccuracies.
- Cloud AI requires network access and service availability.
- API usage may introduce cost or rate limits.
- Local models may require additional memory and setup.
- Personalization is limited without user history and progress data.
- Generated learning resources should be checked before use.

19. FUTURE ENHANCEMENTS

- Voice-based question answering for hands-free study.
- Multilingual interface and multilingual explanations.
- Mobile application for Android and iOS.
- Progress dashboard with topic mastery indicators.
- Gamification using badges, streaks and challenges.
- Teacher/parent dashboards with appropriate privacy controls.
- Moodle or Google Classroom integration.
- Image and PDF input for snapshot-based doubt solving.
- Adaptive learning paths using quiz performance.
- Offline or cached revision mode where technically feasible.

19.1 Proposed Future Architecture

Future EduGenie Ecosystem

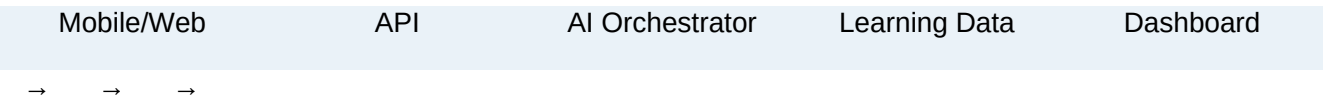


Figure: Future EduGenie Ecosystem

20. CONCLUSION

EduGenie demonstrates how generative AI can be organized into practical educational workflows rather than being used only as an open-ended chatbot. The project combines a FastAPI backend with a simple browser interface and task-specific modules for Q&A, explanation, quiz generation, summarization and learning recommendations.

The expanded implementation in this report adds clearer API contracts, validation patterns, sample Python code, frontend code, prompt templates, pseudocode, testing cases and deployment considerations. These additions provide a foundation for further development while retaining the core architecture and terminology of the supplied EduGenie project.

The project also highlights an important principle for educational AI: generated content should assist learning, while students and educators remain responsible for verifying important information and applying academic judgment.

21. REFERENCES

- Supplied project document: “EduGenie: Google Gemini Powered Learning Assistant.”
- Google AI for Developers documentation (for Gemini API concepts).
- FastAPI documentation (for REST API development).
- Python documentation (for Python language and environment setup).
- Uvicorn documentation (for ASGI application serving).
- Jinja2 documentation (for server-side HTML templating).

TEAM DETAILS

Role			
Team Leader	Team Leader	Sri Dharshini	
Member	Member	Nirmal	
Member	Member	Sriram	
Member	Member	Ashwin	
Member			